# AUTOMATED WEB SECURITY SCANNER

Python-based basic website security configuration checker

| | |
|---|---|
| **Project Type** | Cybersecurity / Python application |
| **Language** | Python |
| **Library** | Requests |
| **Report Version** | 1.0 |

Prepared as documentation for a student cybersecurity project. This report describes the current implementation and the tests performed during development. The scanner performs basic HTTP response and security-header checks; it is not a full penetration-testing or vulnerability-scanning system.

## 1. Introduction

Web applications use HTTPS and HTTP security headers to help protect communication and reduce certain browser-based risks. Checking these settings manually can be repetitive. The Automated Web Security Scanner is a small Python command-line application that accepts a website URL, sends an HTTP request, checks whether the final URL uses HTTPS, inspects five common security headers, and displays findings with recommendations. It also writes scan information to TXT and JSON report files.

## 2. Problem Statement

A user may need a quick way to review a website's basic security configuration without manually inspecting each HTTP response header. This project addresses that limited need by automating a small set of checks and presenting the results in a readable format. It does not attempt to identify every possible vulnerability.

## 3. Objectives

• Validate that the entered value is a complete HTTP or HTTPS URL.

• Send a request to the target and display the HTTP status code and final URL.

• Check whether the final URL uses HTTPS.

• Inspect five common HTTP security headers.

• Show missing-header recommendations and preliminary severity labels.

• Save scan information in TXT and JSON files.

• Handle invalid URLs and request/connection errors without an unhandled program crash.

## 4. Technologies Used

| Technology | Purpose |
|---|---|
| Python | Main programming language and application logic. |

| Technology | Purpose |
|---|---|
| Requests | Sends HTTP requests and reads response status and headers. |
| JSON module | Writes structured scan data to a JSON file. |
| datetime module | Records the scan timestamp. |
| urllib.parse | Parses and validates the basic URL structure. |
| Visual Studio Code / PowerShell | Development environment and command-line execution. |

## 5. System Workflow and Methodology

1. The user starts the program and enters a website URL.

2. The program parses the URL and rejects inputs that do not include a valid HTTP or HTTPS scheme and hostname.

3. The Requests library sends an HTTP GET request with a 10-second timeout and follows redirects.

4. The program records the HTTP status code and final URL, then checks whether the final URL starts with HTTPS.

5. It checks the response for Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, and Referrer-Policy.

6. For each header, the program records whether it is present or missing and provides a recommendation for missing headers.

7. The program saves scan data to scan_report.txt and scan_report.json.

## 6. Security Checks Performed

| Header / Check | Purpose |
|---|---|
| HTTPS | Checks whether the final URL uses HTTPS. This is a basic URL-level check and does not independently audit TLS configuration. |
| Strict-Transport-Security | Helps browsers enforce HTTPS for future connections when correctly configured. |
| Content-Security-Policy | Can help reduce some content-injection risks by restricting permitted content sources. |
| X-Content-Type-Options | Helps prevent browsers from MIME-type sniffing when configured appropriately. |
| X-Frame-Options | Can help reduce clickjacking risks by controlling framing behavior in supporting browsers. |
| Referrer-Policy | Controls how much referrer information browsers send with requests. |

The current code labels missing Strict-Transport-Security and Content-Security-Policy headers as Medium, and the other three missing headers as Low. These are preliminary labels chosen for this project, not a formal risk rating. A missing header does not by itself prove that a site is vulnerable.

## 7. Test Cases and Actual Results

| Test | Input | Observed result | Status |
|---|---|---|---|
| 1. Valid website | https://example.com | HTTP 200; final URL used HTTPS; five headers were reported missing in this run; TXT and JSON reports were saved. | Passed |
| 2. Invalid URL | hello | Displayed the invalid URL message and stopped the scan. | Passed |
| 3. Unreachable hostname | https://this-domain-should-not-exist.invalid | Displayed a DNS/name-resolution request error and continued to save report files. | Passed |

Note: Results reflect the actual development tests shared for this project. The response headers returned by a website may change over time, so results can differ on a later scan.

## 8. Limitations

• The scanner checks only HTTPS usage at the final URL and five response headers.

• It does not perform penetration testing, payload injection, port scanning, authentication testing, or comprehensive TLS analysis.

• A header being present does not guarantee that its value is secure or correctly configured.

• The severity labels are simplified project classifications and should not be treated as a formal vulnerability score.

• Results depend on network access, DNS resolution, redirects, and the target server's response.

• Only scan websites you own or are authorized to assess.

## 9. Conclusion

The Automated Web Security Scanner demonstrates a basic cybersecurity automation workflow using Python. It validates URL input, requests a website response, checks HTTPS usage and five common security headers, provides recommendations for missing headers, and saves scan information in TXT and JSON formats. Tests confirmed that valid input, invalid input, and a connection/DNS error are handled without an unhandled crash. The project can be extended with clearer report status fields, improved output formatting, additional checks, and a graphical web interface. It should be used as an educational configuration checker rather than a replacement for a professional security assessment.

**Ethical use:** Run scans only against systems you own or have explicit permission to assess.